A Appendix

A.1 Figures

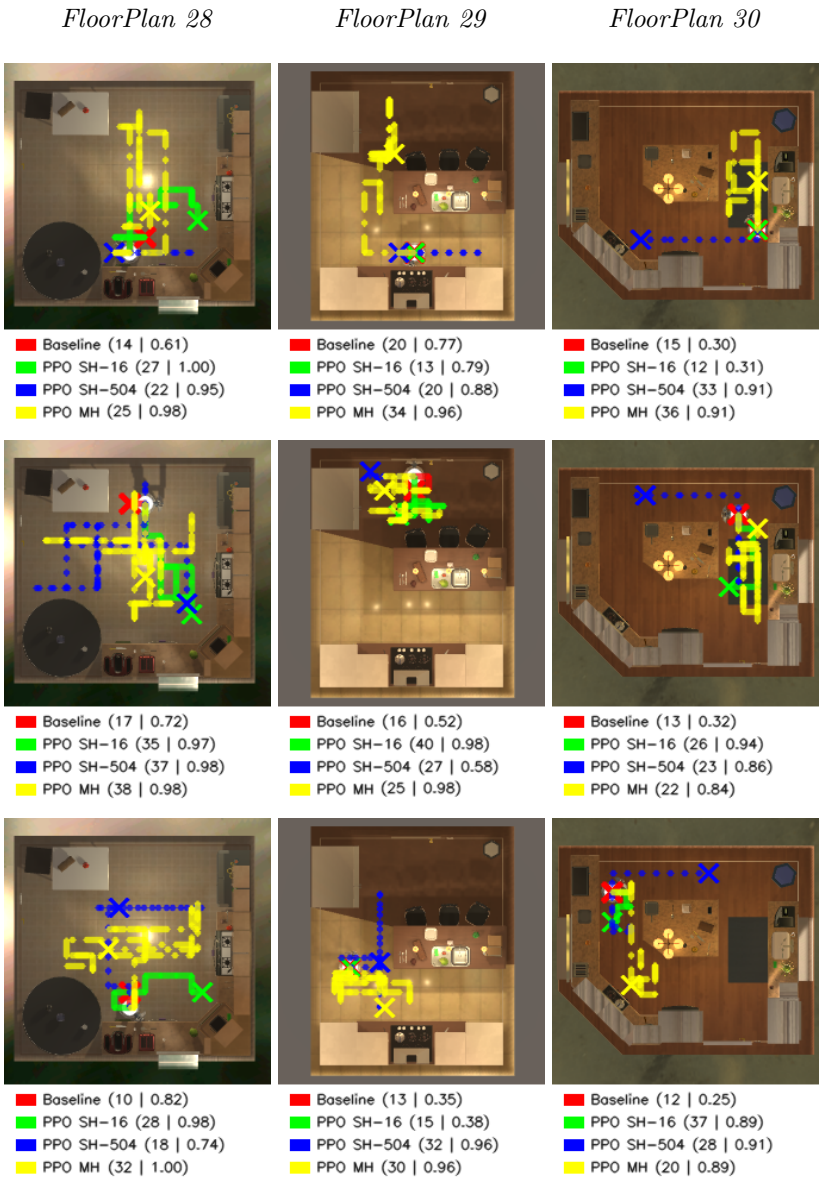

Fig. 5: Representative trajectories on held-out scenes (FloorPlans 28–30). Numbers in parentheses in the legends report `(steps | score)`. All plotted scenarios except the Baseline are with depth input.

A.2 Final Held-Out Metrics

The metrics in Table 1 are computed via a post-hoc evaluation on the held-out scenes (FloorPlans 28–30) using the final training checkpoint of each run (one checkpoint per seed), averaged over episodes and scenes per seed, and then aggregated across seeds.

Table 1: Final held-out performance on FloorPlans 28–30 using the final training checkpoint of each run (means across seeds; Node Recall reported as mean ± std).
Node Recall: fraction of scene graph nodes recalled at episode end.
Episodic Return: cumulative reward under the composite reward function.
Episode Length: number of steps taken until episode termination.
Move Success Rate: fraction of executed translation actions that succeed without collision.
Path Length: total path length traversed during the episode.
↑ indicates that higher values are better.

Scenario	**Node Recall** ↑	**Episodic Return** ↑	**Episode Length**	**Move Success Rate** ↑	**Path Length** ↑
Baseline (R+SH16+IL)	0.48 ± 0.22	0.69	14.0	0.14	0.1
S0 (R+SH16+IL+D)	0.50 ± 0.24	0.69	15.4	0.14	0.2
S1 (R+SH16+D)	0.50 ± 0.23	0.69	16.2	0.15	0.2
S2 (PPO+SH16)	0.58 ± 0.29	0.84	21.0	0.71	1.6
S3 (PPO+SH16+D)	0.58 ± 0.29	0.86	20.8	**0.77**	2.2
S4 (PPO+SH504+D)	0.92 ± 0.14	0.89	32.7	0.42	7.9
S5 (PPO+SH504+D+CL)	0.86 ± 0.19	0.79	28.3	0.30	4.2
S6 (PPO+MH504)	**0.93** ± 0.14	1.07	27.7	0.56	10.8
S7 (PPO+MH504+D)	0.91 ± 0.15	1.04	26.0	0.56	10.1
S8 (PPO+MH504+CL)	**0.93** ± 0.14	1.09	27.8	0.57	12.2
S9 (PPO+MH504+D+CL)	**0.93** ± 0.15	**1.11**	29.3	0.58	**12.9**

A.3 Scenario Overview

To isolate the contribution of individual design choices, we evaluate a set of ablation scenarios that progressively modify the learning setup. The scenarios differ in the RL algorithm, the presence of imitation learning (IL) pretraining, the action parameterisation (single-head vs. multi-head), the use of depth input with a collision-prediction auxiliary head, and whether curriculum learning (CL) with staged action-space expansion is enabled.

Table 2: Overview of ablation scenarios.
IL: imitation learning pretraining.
Actions: single-head parameterisation (either compact 16 actions or enlarged 504 actions) versus factorised multi-head parameterisation (Rotation angle, Length, Stop) at 504 resolution.
Depth w/ Coll.-Aux: depth input with an auxiliary head for collision prediction.
CL: curriculum learning with staged action-space expansion.

Scenario	**RL-Algorithm**	**IL**	**Action Variant**	**Depth w/ Coll.-Aux**	**CL**
Baseline	REINFORCE	✓	Single-Head (16)	–	–
S0	REINFORCE	✓	Single-Head (16)	✓	–
S1	REINFORCE	–	Single-Head (16)	✓	–
S2	PPO	–	Single-Head (16)	–	–
S3	PPO	–	Single-Head (16)	✓	–
S4	PPO	–	Single-Head (504)	✓	–
S5	PPO	–	Single-Head (504)	✓	✓
S6	PPO	–	Multi-Head (504)	–	–
S7	PPO	–	Multi-Head (504)	✓	–
S8	PPO	–	Multi-Head (504)	–	✓
S9	PPO	–	Multi-Head (504)	✓	✓

A.4 Imitation Learning Dataset Generation

Overview and stored format. We generate an offline dataset of expert trajectories in AI2-THOR for imitation learning pretraining of the baseline. Each trajectory is stored as a sequence of samples $\{(x_t, a_t)\}_{t=0}^{T-1}$, where x_t denotes the state representation used during training (RGB, optional depth, local/global scene-graph information, and auxiliary signals such as the previous action), and a_t is a discrete navigation action from the 16-action *single-head* space (8 rotation angles $\times$ 2 lengths, with Stop at rotation angle $= 0$ and length $= 0$). The rotation angles are multiples of 45° and the lengths are $\{0\,\mathrm{m}, 0.3\,\mathrm{m}\}$. For each sample we additionally store metadata (scene identifier, start pose, step index). Every stored trajectory terminates with an explicit Stop action.

Start state selection. For each training scene, we sample multiple start states from the set of reachable positions. A start state is accepted only if movement is possible from that position.

Viewpoint targets and tour ordering. As target structure, we use a precomputed *viewpoint-to-objects* mapping that associates each viewpoint with locally visible objects and their corresponding visibility values. We greedily select a set of viewpoints that maximises the aggregated visibility progress over objects that are not yet sufficiently covered (visibility aggregation follows the multiplicative

update rule explained in Appendix A.6). The selected viewpoints are then ordered into a short tour by solving a travelling salesman problem (TSP) heuristic using Ant-Colony Optimization over the viewpoint positions. The resulting tour is rotated such that it starts at the viewpoint closest to the sampled start state.

Expert action selection (short-horizon planning). The expert follows the current target viewpoint of the tour and selects navigation actions via short-horizon planning. To this end, multiple admissible action candidates are forward-simulated in a separate simulator instance and scored using a heuristic objective that combines (i) visibility gain and first-time crossing of τ ("discovery"), (ii) a bonus for entering previously unvisited grid viewpoints, and (iii) progress towards the target viewpoint. Candidates whose execution fails (e.g., due to blocking) are discarded. In addition, we exclude candidates that perform a rotation but would leave the agent without any valid forward translation afterward (two-step validation consistent with the *move-first* action semantics).

Termination and early stopping. We retain only trajectories that terminate successfully (via STOP) or achieve near-complete graph coverage. In addition to a maximum step budget $T_{\max}$ of 80 steps, we apply early stopping: once at least 95% of ground-truth objects are considered discovered (visibility $\geq \tau$) and no further progress is observed over several steps, we remove stagnating tail steps and append a final STOP action. Expert trajectories are generated offline with a higher step budget ($T_{\max} = 80$) to increase coverage, while online RL episodes remain constrained to 40 steps.

A.5 Stagnation Heuristic

To help the policy decide when further navigation yields diminishing returns, we provide a compact *stagnation signal* that measures how strongly the global semantic scene graph (GSSG) changes over time. Intuitively, if the GSSG embedding barely changes for multiple steps, the agent is likely revisiting already-covered areas, while strong changes indicate ongoing discovery. We encode this signal as an additional state feature used by the navigation policy.

All parameters used are reported in Table 3. We define the change magnitude of the global scene-graph representation as the normalised ℓ_2 difference

$$\Delta_t = \frac{\|\mathbf{g}_t - \mathbf{g}_{t-1}\|_2}{\sqrt{d_{\text{sg}}}}, \tag{1}$$

where $\mathbf{g}_t \in \mathbb{R}^{d_{\text{sg}}}$ denotes the GSSG embedding and d_{sg} is its dimensionality. If no previous embedding is available (sequence start), we set $\Delta_1 := 1$.

For smoothing, we use an exponential moving average (EMA):

$$\bar{\Delta}_t = \alpha \bar{\Delta}_{t-1} + (1 - \alpha)\Delta_t, \tag{2}$$

initialised with $\bar{\Delta}_1 := \Delta_1$. An *embedding change* is detected when $\Delta_t > \tau_{\text{disc}}$, indicating a sufficiently large change in the global scene-graph embedding. We

Table 3: Hyperparameters for stagnation detection.

Parameter	**Value**
EMA smoothing α	0.7
Embedding change threshold τ_{disc}	0.1
Scene-graph dimension d_{sg}	512
Divisor k	10
Stagnation embedding dimension d_{stag}	32

define the indicator

$$d_t = \mathbb{I}[\Delta_t > \tau_{\text{disc}}]\,. \tag{3}$$

To quantify how long the agent has been navigating without producing new semantic information, we track the number of steps since the last embedding change. The counter `steps_since_embedding_change` is denoted by n_t and updated via masking:

$$n_t = (1 - d_t)\, n_{t-1} + 1, \tag{4}$$

with initialisation $n_1 := 1$.

The stagnation signal is then defined as

$$s_t = \tanh\Big(\frac{n_t}{k}\Big)\,, \tag{5}$$

where k is chosen empirically (in our experiments $k = 10$).

As input to the stagnation encoder, we use the concatenation of the three scalar quantities

$$\mathbf{x}_t^{\text{stag}} = \big[\Delta_t,\ \bar{\Delta}_t,\ s_t\big] \in \mathbb{R}^3. \tag{6}$$

The stagnation embedding $\mathbf{e}_t^{\text{stag}} \in \mathbb{R}^{d_{\text{stag}}}$ is then produced by a two-layer MLP:

$$\mathbf{h}_t = \text{ReLU}\big(\mathbf{W}_1\mathbf{x}_t^{\text{stag}} + \mathbf{b}_1\big)\,, \quad \mathbf{W}_1 \in \mathbb{R}^{64\times 3}, \tag{7}$$

$$\mathbf{e}_t^{\text{stag}} = \mathbf{W}_2\mathbf{h}_t + \mathbf{b}_2, \quad \mathbf{W}_2 \in \mathbb{R}^{d_{\text{stag}}\times 64}. \tag{8}$$

A.6 Soft Visibility and Global Visibility Update

We encode object visibility as a continuous score that reflects how reliably an object would be observable from the current viewpoint based on distance and object size. Let d denote the agent–object distance and $s_{\max}$ a size proxy (e.g., maximum 2D/3D extent available from simulator metadata). We compute a soft-visibility score via a sigmoid with a size-dependent centre:

$$V_{\text{soft}}(d, s_{\max}) = \frac{1}{1 + \exp(\omega\,(d - c(s_{\max})))}, \tag{9}$$

where

$$c(s_{\max}) = c_{\text{base}} + k_{\text{size}}\,(s_{\max} - s_{\text{ref}})\,. \tag{10}$$

Here, c_{base} is the baseline visibility range (in metres) for an object of reference size s_{ref}, and k_{size} determines how strongly the sigmoid centre is shifted along the distance axis with object size. The parameter ω determines the steepness of the distance-to-visibility transition. We use empirically selected parameters $\omega = 1.0$, $c_{\text{base}} = 3.5$, $k_{\text{size}} = 1.5$, and $s_{\text{ref}} = 0.5$ that yield a visibility measure consistent with common-sense expectations: larger objects remain visible at larger distances, whereas small objects require closer viewpoints to reach high visibility.

Global aggregation. The GSSG is updated by aggregating local observations over time. Let $V^G_{t-1}(o) \in [0,1]$ denote the global visibility of object o at time $t-1$, and let $V^L_t(o) = V_{\text{soft}}(d_t(o), s_{\max}(o))$ denote the local soft visibility at time t. We update the global visibility with a monotonic, saturating rule: each new observation increases V^G, while the estimate remains bounded in $[0,1]$ and exhibits diminishing returns as V^G approaches 1:

$$V^G_t(o) = 1 - \left(1 - V^G_{t-1}(o)\right)\left(1 - V^L_t(o)\right). \tag{11}$$

A node is counted as discovered if $V^G_t(o) \geq \tau$, where $\tau = 0.8$ is a discovery threshold.

Redundancy control. To reduce redundant updates, we skip the global update for an object if it has already been observed from the same agent position (i.e., repeated rotations at an identical location do not increase visibility). This encourages spatial coverage rather than repeated re-observation from identical viewpoints.

Best-case assumption. Both LSSG extraction and GSSG aggregation are derived from simulator metadata with perfect data association via object identifiers. Consequently, soft-visibility models only *visibility* and not perception failures; the resulting completeness metrics should be interpreted as an idealised best-case reference.

A.7 Relation Extraction from Simulator Metadata

To construct semantic relations in the Local Semantic Scene Graph (LSSG), we derive edges deterministically from AI2-THOR metadata using rule-based geometric and type-specific heuristics, following the oracle-style graph construction protocol of Li et al. in their paper "Embodied Semantic Scene Graph Generation". All relations are computed per time step from the set of currently visible objects and their metadata fields (notably `objectType`, `objectId`, `parentReceptacles`, and the axis-aligned bounding box centre/size).

We generate directed relation pairs with inverse labels where applicable, yielding a multi-relational graph. The relation set comprises:
(i) *support and placement* relations between movable objects and supporting surfaces, inferred from bounding-box overlap in the horizontal plane and small vertical offsets (`supported_by`/`supports`, and `on`/`has_on_top` with optional subtypes such as `standing_on`, `sitting_on`, `lying_on`);

(ii) *hanging* relations for wall-mounted objects, mapped from object categories to relation labels (e.g., `hanging_on`, `pasting_on`, `fixed_on`);
(iii) *relative-position* relations between objects that share the same receptacle, based on 3D proximity and vertical ordering (`close_by`, `above`/`below`);
(iv) *connectivity* relations for adjacent furniture instances of the same type (`connects_to`) and
(v) *attachment/component* relations for small parts that are spatially close to a corresponding main object (`attach_on`/`has_attachment`, `part_of`/`has_part`).

Since relations are derived deterministically from simulator metadata, they are noise-free in our setting and thus represent an idealised upper bound compared to pipelines that infer relations from learned perception and data association.

A.8 Event-based Reward Terms

In addition to the dense potential-based shaping term, we include sparse event-based reward components that encourage translation (especially collision-free translation), reward viewpoint discoveries, and provide an explicit stopping incentive once exploration has stagnated. Coefficients were calibrated on the training scenes to obtain stable learning signals and comparable reward magnitudes, then kept fixed across all ablation scenarios to isolate the effects of algorithmic and action-space changes; held-out evaluation scenes are used only for reporting.

$$r_t^{\text{coll}} = -0.02 \cdot \mathbb{I}[\texttt{move_failed} \vee \texttt{actual_dist} < \texttt{target_dist} - 0.05]\,, \quad (12)$$

$$r_t^{\text{move}} = 0.005 \cdot \mathbb{I}[\texttt{actual_dist} > 0.05]\,, \quad (13)$$

$$r_t^{expl} = 0.01 \cdot \mathbb{I}[\Delta|\mathcal{O}_{seen}| > 0]\,, \quad (14)$$

$$r_t^{stop} = b_{stop} \cdot \mathbb{I}[a_t = \text{Stop} \wedge \texttt{steps_since_exploration} \geq K]\,, \quad (15)$$

Here, `target_dist` denotes the translation distance requested by the action, `actual_dist` is the distance actually traveled in the environment, $\mathcal{O}_{seen}$ denotes the set of object identifiers that have been observed at least once so far (i.e., the set of keys in the viewpoint-tracking map), `move_failed` indicates a failed motion step (e.g., due to collision or blocking), and `steps_since_exploration` denotes the number of steps since the last first-time object observation event.

A.9 Adaptive Curriculum for Large Discrete Action Spaces

All relevant hyperparameters are listed in Table 4. We use a staged curriculum to ease optimisation in large discrete action spaces. The curriculum is applied to both the enlarged single-head action space (504 actions) and the factorised multi-head parameterisation (Rotation angle, Length, Stop) at the same 504-action resolution. In each stage, only a restricted subset of actions (single-head) or action-factor combinations (multi-head) is admissible, and invalid choices are masked by setting their logits to $-\infty$.

Table 4: Curriculum learning hyperparameters for large action spaces.

Parameter	**Value**
Min. stage blocks	50
Plateau evaluation window W (blocks)	50
Recent window R (blocks)	10
Plateau threshold	0.02
Forced-promotion backstop (blocks)	250
Entropy boost factor	2.0
Entropy boost duration (blocks)	20
EMA smoothing α	0.2

Stages. We progressively increase action-space resolution over four stages (Table 5). In the multi-head parameterisation, stages restrict admissible rotation-angle and length indices. In the single-head parameterisation, stages restrict admissible atomic actions to the corresponding Cartesian product. The STOP decision remains available throughout. Stage 3 uses a near-complete 0.1 m grid; we exclude 1.9 m to keep the stage at 20 lengths while ensuring the length set grows monotonically and contains all Stage 2 values.

Table 5: Curriculum stages for large action spaces (STOP available throughout). Lengths in metres.

Stage	**#Angles**	**Angle step**	**#Lengths**	**Total**	**Lengths**
1	8	45°	2	16	$\{0.0, 0.3\}$
2	8	45°	6	48	$\{0.0, 0.3, 0.7, 1.2, 1.6, 2.0\}$
3	8	45°	20	160	$\{0.0, 0.1, \ldots, 2.0\} \setminus \{1.9\}$
4	24	15°	21	504	$\{0.0, 0.1, \ldots, 2.0\}$

Plateau detection. To decide when to promote to the next stage, we track block-aggregated rollout statistics and smooth them via an exponential moving average (EMA) with smoothing factor α (Table 4) to reduce noise. Specifically, we monitor (i) the primary task score (Node Recall), (ii) the mean episodic return, and (iii) the mean episode length (steps), since these three signals jointly capture coverage progress, optimisation progress under the shaped reward, and termination behaviour. Plateau decisions are evaluated every W blocks, where W is the *plateau evaluation window*. For each metric, we compare the mean value over the most recent R blocks (the *recent window*) against the mean value over the preceding $W-R$ blocks (the *earlier window*) on the smoothed curves. A promo-

tion is triggered if the relative improvement between recent and earlier window is below the plateau threshold for all monitored metrics.

Backstop and minimum stage duration. To prevent the curriculum from stalling indefinitely, we (i) enforce a minimum number of blocks spent in each stage and (ii) apply a backstop that promotes after a fixed block budget even if plateau conditions are not met.

Entropy ramp after promotion. After promotion, the entropy coefficient is temporarily increased and then linearly annealed back to its default value over a fixed number of blocks. This encourages the policy to explore the newly available actions rather than remaining confined to the previous stage's behaviour.

A.10 Collision Auxiliary Head and Loss

The collision head is trained as a binary classifier and estimates, per time step, the probability that the last *translation* step could not be completed due to blockage or collision. As supervision, we use the environment's binary motion feedback (success vs. failure) and define a label $y_t \in \{0, 1\}$ (1: collision/blockage, 0: successful motion).

Importantly, we supervise the collision head *only* on actions with non-zero translation length. Pure rotation steps (i.e., length $= 0$) are masked out and do not contribute to the loss. If a mini-batch contains no translation actions, the collision loss is set to zero.

We optimise binary cross entropy with logits on the head logits z_t:

$$\mathcal{L}_{\text{collision}} = \frac{1}{\sum_{t=1}^{T} m_t} \sum_{t=1}^{T} m_t \cdot \text{BCEWithLogits}(z_t, y_t),$$

where $m_t \in \{0, 1\}$ is a mask indicating whether step t is a translation action (1) or a rotation-only action (0).

A.11 Optuna Objective

To select hyperparameters in a reproducible and task-aligned manner, we optimise a single scalar objective that prioritises scene-graph completeness while applying light secondary penalties for collisions, truncations, and overly long episodes on a recent rollout window.

$$J = \bar{s} - 0.20\,\bar{c} - 0.10\,\bar{\tau} - 0.01\,\max(0,\, \bar{n} - 25)\,, \tag{16}$$

where $\bar{s}$ is the mean episode score (Node Recall) within the considered rollout window, $\bar{n}$ is the mean episode length (in steps), $\bar{\tau}$ is the truncation rate (fraction of episodes terminated by the step limit), and $\bar{c}$ is the mean collision rate in the same window. The window spans the most recent ≈ 50 training blocks. For numerical stability, we clip the objective from below at -100.

Table 6: Per-trial training budget and pruning configuration used during hyperparameter optimisation. PPO entries listed as "SH16 / SH504 / MH" correspond to the compact single-head action space (16 actions), the full single-head action space (504 actions), and the multi-head factorised action space (24 rotation angles, 21 lengths, STOP), respectively.

Scenario	Blocks / trial	Warmup (blocks)	Pruner	Startup trials
REINFORCE+IL	400	200	Median	2
PPO SH16	400	200	Median	2
PPO SH504	800	600	Percentile (25%)	3
PPO MH	500	300	Median	2

Optimisation budget and pruning. Hyperparameters are optimised with Optuna using a TPE sampler (5 startup trials). Each trial trains a policy for a fixed number of blocks, depending on the scenario, to account for different learning speeds (Table 6). Early stopping is implemented via Optuna pruning: we report J at block resolution and prune underperforming trials after a warmup phase. For REINFORCE+IL, PPO with compact actions and multi-head large action space, we use a MedianPruner; for the slower-learning single-head large action space we use a less aggressive PercentilePruner (bottom 25%). In all cases, pruning is checked every block and only enabled after the respective warmup; pruners are additionally activated only after a small number of completed trials (startup trials), following Optuna defaults. All Optuna tuning runs are performed with depth and without curriculum learning.

Table 7: Optuna hyperparameter search spaces. All parameters were optimised over categorical candidate sets. The optimisation target is the composite objective in Eq. 16.

Parameter	**Search Space (categorical)**	**Best**
REINFORCE (all setups)		
α	{1e-5, 3e-5, 5e-5, 1e-4, 5e-4}	**5e-4**
γ	{0.90, 0.95, 0.97, 0.99}	**0.97**
c_{aux} (collision)	{0.2, 0.3, 0.4}	**0.3**
c_{ent} (entropy)	{0.01, 0.05, 0.10, 0.15}	**0.05**
PPO (Single-Head, 16 actions)		
α	{1e-5, 3e-5, 5e-5, 1e-4, 5e-4}	**1e-4**
ϵ (clip)	{0.10, 0.15, 0.20, 0.25, 0.30}	**0.25**
λ (GAE)	{0.90, 0.93, 0.95, 0.97, 0.98}	**0.90**
c_{aux} (collision)	{0.2, 0.3, 0.4}	**0.2**
c_{ent} (entropy)	{0.01, 0.05, 0.10, 0.15}	**0.05**
PPO (Single-Head, 504 actions)		
α	{1e-5, 3e-5, 5e-5, 1e-4, 5e-4}	**1e-4**
ϵ (clip)	{0.10, 0.15, 0.20, 0.25, 0.30}	**0.15**
λ (GAE)	{0.90, 0.93, 0.95, 0.97, 0.98}	**0.97**
c_{aux} (collision)	{0.2, 0.3, 0.4}	**0.3**
c_{ent} (entropy)	{0.01, 0.05, 0.10, 0.15}	**0.05**
PPO (Multi-Head, 504 resolution)		
α	{1e-5, 3e-5, 5e-5, 1e-4, 5e-4}	**1e-4**
ϵ (clip)	{0.10, 0.15, 0.20, 0.25, 0.30}	**0.15**
λ (GAE)	{0.90, 0.93, 0.95, 0.97, 0.98}	**0.97**
c_{aux} (collision)	{0.2, 0.3, 0.4}	**0.3**
c_{ent} (entropy)	{0.01, 0.05, 0.10, 0.15}	**0.05**

A.12 Further Tables

Table 8: Feature dimensions and architecture specifications.

Component	**Parameter**	**Value**
Input dimensions		
	RGB Image	224×224×3
	Depth Map (optional)	224×224×1
Feature Extractors		
	RGB encoder (ResNet18)	512-dim
	Depth CNN layers	3
	Depth encoder output	256-dim
	Action embedding	64-dim
	Stagnation embedding	32-dim
Graph encoder (HGT)		
	LSSG embedding	512-dim
	GSSG embedding	512-dim
	HGT hidden dimension	128
	HGT attention heads	4
	HGT layers	2
Policy core		
	Concatenated state vector w/ depth	1888-dim
	Concatenated state vector w/o depth	1632-dim
	LSTM layers	2
	LSTM hidden units	1024
Action space — Multi-Head (Variant B)		
	Rotation angles	24 (0–345° in 15° steps)
	Lengths	21 (0.0–2.0 m in 0.1 m steps)
	Stop head	2 (binary)
	Motion resolution	504
Action space — Single-Head (Variant A)		
	Compact action set	16 (8 rots × 2 lens)
	Enlarged action set	504 (24 rots × 21 lens)
Curriculum stages		
	Stage 1	16 (8 rots × 2 lens)
	Stage 2	48 (8 rots × 6 lens)
	Stage 3	160 (8 rots × 20 lens)
	Stage 4	504 (24 rots × 21 lens)

Table 9: Hyperparameters for training and reward shaping. PPO entries listed as "SH16 / SH504 / MH" correspond to the compact single-head action space (16 actions), the full single-head action space (504 actions), and the multi-head factorised action space (24 rotation angles, 21 lengths, STOP), respectively.

Category	**Parameter**	**Value**
PPO hyperparameters (SH16 / SH504 / MH)		
	Learning rate α	1e-4 / 1e-4 / 1e-4
	Discount factor γ	0.99
	GAE lambda λ	0.90 / 0.97 / 0.97
	Clipping parameter ϵ	0.25 / 0.15 / 0.15
	Value loss coefficient c_v	0.5
	Entropy coefficient c_{ent}	0.05 / 0.05 / 0.05
	Collision loss coefficient c_{aux}	0.2 / 0.3 / 0.3
	Epochs per update K	4
	Gradient clipping (max_norm)	0.5
	KL target (early stopping)	0.01
REINFORCE hyperparameters		
	Learning rate α	5e-4
	Discount factor γ	0.97
	Entropy coefficient c_{ent}	0.05
	Collision loss coefficient c_{aux}	0.3
Reward shaping		
	Node weight λ_{node}	0.1
	Visibility weight λ_p	0.5
	Viewpoint diversity weight λ_d	0.001
	Edge visibility P_{edge}	1.0
	Time penalty ρ	0.001
	Collision penalty (event term)	-0.02
	Movement bonus (event term)	$+0.005$
	Exploration bonus (event term)	$+0.01$
	Stop bonus (event term)	0.05
	Min. steps for stop bonus K	5
	Visibility threshold (Node Recall)	0.8